

export class InputBox

- [export class InputBox](#)
 - [Construtor](#)
 - [Método parseTemplate](#)
 - [Método onOkButtonClick](#)
 - [Método showInputBox](#)
 - [Método inputBox](#)
 - [REFERÊNCIAS](#)
-

Construtor

- **Descrição**
 - O construtor da classe **InputBox** inicializa uma instância para criar uma caixa de entrada com campos definidos em um modelo de texto.
 - Ao criar uma nova instância de InputBox, o construtor executa as seguintes operações:
 1. Inicialização dos Campos: A propriedade fields é definida como uma lista vazia ([]), que será preenchida posteriormente com os campos processados pelo método parseTemplate.
 2. Armazenamento do Título: Atribui o valor de title ao atributo title da instância, para ser exibido no cabeçalho da caixa de entrada.
 3. Processamento do Modelo (template): O modelo é dividido em linhas, e o método parseTemplate é chamado para processá-las, populando fields com os dados dos campos.
 4. Inclusão Dinâmica de CSS: Verifica se o arquivo de estilo input_Box.css já está incluído no documento. Caso não esteja, ele é adicionado dinamicamente com o método includeCSS da classe MiMethods.
- **Parâmetros**
 1. title (string): Define o título da caixa de entrada. Este valor é armazenado na propriedade title.
 2. template (string): Modelo de texto contendo a lista de campos, onde cada linha representa um campo de entrada.
- **Sintaxe**

```
const title = 'Cadastro';
const template = `
    ~Nome do Aluno:~\\ssssssssssssssssssssssssssss
^Bnome_do_aluno ~Número:      ~\\ssssss^Bnumero
    ~Endereço:      ~\\ssssssssssssssssssssssssssss^Bendereco
~nome da mãe:~\\ssssssssssssssssssss^Bnome_da_mae
                \\
~Idade:      ~\\sss^Bidade
`;

const inputBox = new InputBox(title, template);//Executa constructor(title, template)
```

- **Observações**

- O arquivo CSS `input_Box.css` precisa estar localizado no caminho relativo `../css/input_Box.css`.
- O método `parseTemplate` é responsável por interpretar o template e adicionar os campos à propriedade `fields`.
- O método `MiMethods.includeCSS` deve estar implementado no projeto para permitir a inclusão dinâmica do CSS.

Método `parseTemplate`

- **Descrição:**

Este método analisa uma string de template e extrai informações sobre os campos a serem preenchidos, construindo uma estrutura de dados que descreve cada campo.

- **Parâmetros:**

- `template` (string): A string de template a ser analisada.

- **Retorno:**

- Um array de arrays, onde cada array interno representa uma linha do template e contém objetos que descrevem os campos daquela linha.

- **Estrutura de Dados de Retorno:**

- Cada objeto dentro do array de linhas possui as seguintes propriedades:
 - `label` (string): O rótulo do campo.
 - `maxLength` (number): O tamanho máximo do valor do campo.
 - `align` (string): O alinhamento do campo ('left', 'right' ou 'center').

- **Lógica do Método:**

1. **Divisão em Linhas:** A string de template é dividida em linhas para facilitar a análise.
2. **Análise de Caracteres:** Cada caractere da linha é analisado:

- `~`: Alterna entre a definição do rótulo e do valor.
- `\`: Indica o início de um campo e o número de caracteres que ele pode conter.
- Outros caracteres: São adicionados ao rótulo se estivermos dentro de uma seção de rótulo.

3. **Construção da Estrutura de Dados:** À medida que os campos são identificados, seus detalhes são adicionados a um objeto e este objeto é adicionado ao array de linhas.
4. **Alinhamento:** O alinhamento do campo é determinado pela presença de dois pontos (`:`) no rótulo.

- **Exemplo de Uso:**

```
const inputBoxTemplate =
`
  ~Nome do Aluno:~\\ssssssssssssssssssssssssssssss
^Bnome_do_aluno ~Número:      ~\\sssssss^Bnumero
  ~Endereço:      ~\\ssssssssssssssssssssssssssss^Bendereco
~nome da mãe:~\\ssssssssssssssssssss^Bnome_da_mae
              \\
~Idade:        ~\\sss^Bidade
`;

const parser = new Parser(); // Assumindo que 'Parser' é a classe
que contém este método
parser.parseTemplate(inputBoxTemplate);

console.log(parser.fields);
```

Método **onOkButtonClick**

- **Descrição**

- Este método, **onOkButtonClick**, pertence a uma classe JavaScript (não nomeada no exemplo) e é usado para coletar os valores dos campos em um formulário estruturado, organizar os dados em um objeto e retornar esses valores quando o botão "OK" é clicado. O método fecha a caixa de diálogo (representada pelo **overlay**) e resolve uma promessa com os dados coletados.

- **Assinatura do Método**

```
onOkButtonClick(overlay, resolve)
```

- **Parâmetros**

- **overlay**: Elemento do tipo `HTMLDivElement`, que representa a camada de sobreposição (caixa de diálogo) a ser removida da página ao concluir a coleta dos dados.
- **resolve**: Função de callback para resolver uma promessa com os valores coletados dos campos do formulário.

- **Fluxo de Execução**

1. **Inicialização de `inputValues`**: Um objeto vazio é criado para armazenar os valores dos campos do formulário, onde a chave será o `name` do campo e o valor será o valor do campo.
2. **Função `processField`**:
 - Recebe um campo, índice da linha e índice do campo como parâmetros.
 - Define um `id` padrão para o campo caso ele não tenha um `id` definido.
 - Obtém o elemento `input` correspondente no DOM.
 - Se o campo existe e possui um `name` definido, armazena o valor no objeto `inputValues` com a chave sendo o `name`.
3. **Função `processLine`**:
 - Recebe uma linha de campos e seu índice, então itera sobre os campos chamando `processField` para processar cada um deles.
4. **Iteração sobre as linhas**:
 - Utiliza `this.fields` (assumido como uma matriz de linhas do formulário), iterando sobre cada linha e chamando `processLine`.
5. **Fechamento e Resolução**:
 - Remove o elemento `overlay` do DOM.
 - Chama `resolve(inputValues)` para retornar os valores coletados.

- **Exemplo de Uso**

- Este método é tipicamente chamado quando um botão de confirmação (como "OK") é pressionado em uma caixa de diálogo. Pode ser utilizado em contextos onde a coleta e envio dos dados do formulário são essenciais para a próxima etapa do fluxo.

- **Exemplo de Código**

```
// Exemplo de chamada ao método onClick:
const overlay = document.createElement('div'); // Exemplo de
// elemento de sobreposição
const fields = [
  [{ id: 'campo1', name: 'nome' }],
  [{ id: 'campo2', name: 'email' }]
];

const myClassInstance = new MyClass(fields); // Presume-se que a
// classe contenha o método onClick

myClassInstance.onClick(overlay, (data) => {
  console.log('Dados coletados:', data);
});
```

- Considerações Adicionais
 - **Modularidade:** A divisão em funções auxiliares (`processField` e `processLine`) torna o código mais modular e facilita a manutenção.
 - **Flexibilidade do Campo:** O método garante que cada campo tenha um `id` e um `name`, proporcionando mais flexibilidade na estrutura de dados e evitando erros de referência.

Método `showInputDialog`

- Cria uma caixa de diálogo com base no array de campos `this.fields`, exibindo um formulário dinâmico com os campos especificados. Esse método é útil para coletar dados do usuário e possui botões de confirmação e cancelamento.
- **Descrição**
 - O método `showInputDialog` cria uma sobreposição (`overlay`) com uma caixa de diálogo contendo:
 1. Um título.
 2. Campos dinâmicos definidos no array `this.fields`.
 3. Botões de confirmação ("OK") e cancelamento ("Cancelar").
 - Quando o usuário confirma (clcando em "OK"), a função `resolve` é chamada para capturar os dados. Ao cancelar, a função retorna `null`.
- **Parâmetros**
 - **resolve:** `Function` — Função que recebe o resultado da interação do usuário com a caixa de diálogo. Resolve com os dados preenchidos se "OK" for clicado, ou `null` se "Cancelar" for escolhido.
- **Fluxo de Execução**
 1. **Criação do overlay:** Uma `div` de sobreposição (`overlay`) é criada para bloquear a interface de fundo enquanto a caixa de diálogo é exibida.

2. Construção da caixa de diálogo:

- Cria o elemento `form` com a classe `inputBox-dialog-box` para estilização da caixa de diálogo.

3. Configuração do título:

- O título do diálogo é inserido no elemento `h2`, utilizando o valor `this.title`.

4. Instanciação do contêiner de campos:

- Um objeto `FormFieldsContainer` é criado para gerenciar os campos de entrada.
- Para cada elemento em `this.fields`, adiciona uma linha de campo no contêiner.
- Ajusta a largura das colunas com `adjustColumnWidths`.

5. Adição de botões de controle:

- Cria uma `div` para conter os botões.
- Cria um botão **OK**:
 - Ao clicar, chama o método `onOkButtonClick` que remove o `overlay` e chama `resolve` com os dados preenchidos.
- Cria um botão **Cancelar**:
 - Ao clicar, remove o `overlay` e chama `resolve` com `null`.

6. Adição à árvore DOM:

- Insere o contêiner de campos e o contêiner de botões na caixa de diálogo `form`.
- Insere o formulário na sobreposição `overlay` e o adiciona ao `body` da página.

• Estilização

◦ Classes CSS utilizadas:

- `inputBox-dialog-overlay`: para a sobreposição de fundo.
- `inputBox-dialog-box`: para a caixa de diálogo.
- `inputBox-dialog-button-container`: para o contêiner dos botões.

• Ver Também

- Método `onOkButtonClick`: Processa a confirmação do usuário.
- Classe `FormFieldsContainer`: Manipula e estrutura os campos de entrada na caixa de diálogo.

• Exceções

- Nenhuma exceção específica é tratada neste método. A remoção do `overlay` e a resolução de `resolve` garantem que o diálogo sempre será descartado corretamente.

• Exemplo de Uso

```
const dialog = new MyDialogClass(fieldsArray, title);
dialog.showInputBox((result) => {
  if (result !== null) {
    console.log("Dados recebidos:", result);
  } else {
    console.log("Usuário cancelou a entrada de dados.");
  }
});
```

Método **inputBox**

Descrição:

Este método estático cria uma nova instância de **InputBox** e exibe uma caixa de diálogo para entrada de dados, retornando uma **Promise** que é resolvida com os valores de entrada fornecidos pelo usuário.

Parâmetros:

1. **title** (string): O título que será exibido na caixa de diálogo.
2. **template** (string): O template que define os campos a serem preenchidos na entrada de dados.

Retorno:

Uma **Promise** que é resolvida com um objeto contendo os valores de entrada do usuário quando ele clica no botão "OK" ou **null** se o usuário cancelar a operação.

Lógica do Método:

1. **Criação da Instância:** Cria uma nova instância da classe **InputBox**, passando o **title** e o **template** como argumentos.
2. **Exibição da Caixa de Diálogo:** Chama o método **showInputBox** da instância de **InputBox**, passando a função **resolve** para que os valores de entrada possam ser retornados quando o usuário interagir com a caixa.

Exemplo de Uso:

```
InputBox.inputBox('Título da Entrada', 'Template de Campos').then((inputValues) => {
  if (inputValues) {
    console.log('Valores de entrada:', inputValues);
  } else {
    console.log('Entrada cancelada pelo usuário.');
```

REFERÊNCIAS

1. [Chatgpt](#)
2. [Código fonte](#)